Unified Digital Wellness System

A PROJECT REPORT

Submitted by

Abhijeet Sandilya (24MIM10074)

Ayush Chandravanshi (24MIM10221)

Atharva Padole (24MIM10038)

Soma Vashista (24MIM10098)

in partial fulfillment for the award of the degree

of

INTEGRATED MASTER OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL INTELLIGENCE (SCAI)

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

APRIL , 2026

VIT BHOPAL UNIVERSITY, KOTHRIKALAN,SEHORE

MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

This is to certify that the thesis titled AI-Driven Student Well-Being Monitoring System, submitted by Abhijeet Sandilya (24MIM10074), Ayush Chandravanshi (24MIM10221) , Atharva Padole (24MIM10038), and Soma Vashista (24MIM10098) to VIT Bhopal University, in partial fulfillment of the requirements for the award of the degree of **Integrated Master of Technology in Artificial Intelligence**, is a bona fide work carried out under my supervision. The thesis fulfills the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this thesis have not been submitted, and will not be submitted, either in part or in full, for the award of any other degree or diploma, and the same is certified.

Supervisor

Signature
Name: Dr Jyoti Chauhan
Designation: Assistant Professor
Date:

Program Chair

Signature
Name: Dr.S Devaraju
Designation: Assistant Professor
Date

Reviewer 1

Signature
Name: Dr.S.Periyanayagi
Designation: Assistant Professor
Date:

Reviewer 2

Signature
Name: Dr.Abha Trivedi
Designation: Assistant Professor
Date

A viva-voce was contucted on

ACKNOWLEDGEMENT

I am deeply indebted to **Dr. G. Viswanathan**, Chancellor of VIT Bhopal University, and **Dr. T.B. Sridharan**, Pro Vice-Chancellor, VIT Bhopal University, for their unwavering support and guidance. Their leadership and vision have been a constant source of inspiration throughout my academic journey. Express my profound gratitude to **Dr. Pon Harshavardhan**. Dean of SCAl, for his encouragement and invaluable support at every stage, which significantly contributed to the successful completion of my project and degree. My heartfelt thanks go to **Dr. S. Devaraju**, Program Chair of Integrated M.TechArtificial Intelligence, and **Dr. Venkat Prasad Padhy**, Division Head of the Al Division, for their timely assistance, guidance, and invaluable insights during the course of my research.

A special note of appreciation reserved for my supervisor, **Dr Jyoti Chauhan**, whose constant support, patience, and expertise were instrumental in completing this project. Her guidance, encouragement, and clarity in addressing my queries made this journey both meaningful and enriching.

I would like to express my sincere gratitude to **Dr.S.Periyanayagi** and **Dr.Abha Trivedi**. for their valuable feedback and insightful reviews.Lastly, I extend my deepest gratitude to my parents and friends, whose unwavering encouragement and support have been my driving force throughout this research endeavor. Their belief in me has been a source of strength and motivation.

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BGR	Blue-Green-Red (OpenCV colour format)
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
CV	Computer Vision
DWS	Digital Wellness System
EAR	Eye Aspect Ratio
FPS	Frames Per Second
GUI	Graphical User Interface
HTTP	HyperText Transfer Protocol
JPEG	Joint Photographic Experts Group
LLM	Large Language Model
MAR	Mouth Aspect Ratio
MJPEG	Motion JPEG
MTV	Model-Template-View
MVC	Model-View-Controller
ORM	Object-Relational Mapper
RAG	Retrieval-Augmented Generation
RGB	Red-Green-Blue
SQLite	Structured Query Language Lite (embedded DB)
TTS	Text-to-Speech
UI	User Interface
URL	Uniform Resource Locator
WSGI	Web Server Gateway Interface

Figures and Graphs

Tables

Abstract

This project presents the Smart Health Monitor, a full-stack AI-driven web application designed to promote the physical wellness of users who spend extended hours in front of digital screens. The system addresses three interrelated domains of digital wellness: posture monitoring during work, yoga-guided physical activity during leisure, and conversational health analytics powered by a Retrieval-Augmented Generation (RAG) chatbot.

The Weekday Mode employs Google's MediaPipe Face Mesh and Pose Landmark detection pipelines to continuously analyse the user's webcam feed in real time. It computes the Eye Aspect Ratio (EAR) for blink and drowsiness detection, applies geometric comparisons of skeletal landmark positions to identify poor posture states (slouching, forward head, tilted shoulders, excess screen proximity), and delivers immediate voice notifications via pyttsx3 when deviations persist beyond configurable thresholds.

The Weekend Mode leverages the MediaPipe Pose Landmark model to classify seven standard yoga asanas — T-Pose, Virabhadrasana II, Vrikshasana, Adho Mukha Svanasana, Uttanasana, Utkatasana, and Urdhva Hastasana — using joint angle geometry computed at runtime from 33 three-dimensional pose landmarks. A pose-stability counter and a hold-timer overlay provide real-time feedback, guiding users through each asana.

All monitoring sessions are persisted using Django's ORM against an SQLite database, recording duration, blink counts, bad-posture time, and yoga duration. A comprehensive Analytics Dashboard aggregates these records into actionable statistics and personalised health insights. A domain-restricted RAG Chatbot, built on LangChain with Ollama's llama3.2 model and a Chroma vector store, allows users to query their own wellness history in natural language.

The system is engineered on the Django MTV framework with OpenCV for video acquisition, multipart HTTP streaming for the live camera feed, and a thread-safe camera management layer that guarantees clean release of hardware resources. Light-mode and dark-mode themes, responsive navigation, and session-history views complete the user experience. The system is locally deployable, privacy-preserving, and requires no specialised hardware beyond a standard webcam.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Abbreviations List of Figures and Graphs List of Tables Abstract	
1	**CHAPTER-1:** **PROJECT DESCRIPTION AND OUTLINE** 1.1 Introduction 1.2 Motivation for the Work 1.3 Introduction to Techniques Used 1.4 Problem Statement 1.5 Objectives of the Work 1.6 Organization of the Project 1.7 Summary	
2	**CHAPTER-2 :** **RELATED WORK INVESTIGATION** 2.1 Introduction 2.2 Existing Approaches and Methods *2.2.1 Real-Time Facial Landmark Detection* *2.2.2 EAR-Based Drowsiness and Blink Detection* *2.2.3 Yawning Detection Using Mouth Metrics* *2.2.4 Web-Based Health Monitoring Applications* 2.3 Pros and Cons of Existing Approaches 2.4 Issues and Observations from Investigation 2.5 Summary	
3	**CHAPTER-3 :** **REQUIREMENT ARTIFACTS** 3.1 Introduction 3.2 Hardware and Software Requirements 3.3 Specific Project Requirements *3.3.1 Data Requirements* *3.3.2 Functional Requirements* *3.3.3 Performance and Security Requirements* *3.3.4 Look and Feel Requirements* 3.4 Summary	
4	**CHAPTER-4:** **DESIGN METHODOLOGY AND ITS NOVELTY** 4.1 Methodology and Objective 4.2 Functional Module Design and Analysis 4.3 Software Architectural Design 4.4 Subsystem Services 4.5 User Interface Design 4.6 Summary	

	CHAPTER-5 TECHNICAL IMPLEMENTATION AND ANALYSIS 5.1 Description 5.2 Technical Coding and Code Solutions 5.2.1 Django Project Structure 5.2.2 MediaPipe Face Landmarker Integration 5.2.3 Eye Aspect Ratio (EAR) Computation 5.2.4 Mouth Aspect Ratio (MAR) Computation 5.2.5 Blink Rate Computation 5.2.6 pyttsx3 Alert Engine 5.3 Algorithm 5.4 Working Layout of Forms 5.5 Prototype Submission 5.6 Testing and Validation 5.7 Performance Analysis 5.8 Summary	

CHAPTER-1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

The 21st-century lifestyle has precipitated a fundamental shift in human behaviour, characterised by an unprecedented reliance on digital screens. The convergence of remote work culture, online education, social connectivity, and digital entertainment has resulted in individuals, particularly students and knowledge professionals, spending the majority of their waking hours in front of computers, tablets, and smartphones. What began as a temporary adaptation during the global pandemic has solidified into a permanent mode of operation, with studies indicating that the average knowledge worker now spends upwards of 13 hours per day engaging with digital devices. This prolonged engagement, while enabling productivity and connectivity, has introduced a new class of occupationally induced health challenges that the fields of ergonomics and preventive medicine are only beginning to fully comprehend.

The physiological toll of this digital immersion manifests across multiple dimensions of health. Musculoskeletal disorders, collectively known as "computer posture syndrome," arise from sustained static loading of the cervical and lumbar spine, leading to chronic conditions such as forward head posture, rounded shoulders, and lower back pain. Ophthalmic health is compromised through a condition termed Computer Vision Syndrome (CVS), characterised by reduced blink frequency, digital eye strain, dry eyes, and accommodative dysfunction. Furthermore, the cognitive and neurological impacts of prolonged screen exposure, including mental fatigue, reduced attention spans, and disruptions to circadian rhythms through blue light exposure, constitute a significant but often under-addressed aspect of digital wellness. This constellation of ailments is no longer merely an inconvenience; it represents a systemic public health challenge that has the potential to diminish long-term quality of life and workforce productivity.

Conventional approaches to mitigating these risks have predominantly relied on simplistic, passive tools. Timer-based applications, such as the Pomodoro technique or break

reminders, operate on a fixed schedule, failing to account for the user's actual physiological state or the dynamic nature of their posture. Manual self-assessment is notoriously unreliable, as individuals seldom possess the proprioceptive awareness or objective metrics to detect subtle but harmful postural deviations. Commercial ergonomic assessments, while effective, are often prohibitively expensive and available only to a fraction of the workforce. More sophisticated solutions, such as wearable sensors or depth-sensing cameras, offer precise data but introduce barriers of cost, convenience, and user compliance. Critically, almost all existing solutions treat data in isolation: they may remind a user to stretch, but they do not correlate that reminder with the user's blink rate from the previous hour, nor do they provide a historical narrative of the user's wellness journey.

The Smart Health Monitor — Unified Digital Wellness System is conceived to address this critical gap. It is not simply a posture reminder or a yoga guide; it is an integrated, AI-driven digital wellness ecosystem designed to function as a continuous, ambient health companion. The project leverages the paradigm of "ubiquitous computing," embedding intelligence into the user's existing environment—their standard webcam and personal computer—to provide real-time, personalised, and actionable health insights. By harnessing the power of Google's MediaPipe computer vision pipelines, the system transforms a commodity webcam into a sophisticated biometric sensor, capable of extracting high-fidelity physiological data (3D facial and body landmarks) without any specialised hardware.

The system's architecture is rooted in a deep understanding of the user's lifestyle rhythm. It operates under two primary, context-aware modes that mirror the cyclical nature of work and recovery. Weekday Mode functions as a continuous digital ergonomics monitor, analysing the user's posture and eye behaviour in real-time during periods of focused work. It moves beyond simple binary alerts, offering nuanced detection of five distinct categories of poor posture—slouching, forward head carriage, shoulder tilt, head tilt, and excessive screen proximity—and correlates this with blink rate analysis to identify patterns of fatigue. Weekend Mode recontextualises the same hardware to facilitate active recovery through guided physical activity. It introduces a novel, training-free approach to yoga pose classification, using geometric joint-angle analysis to recognise and provide real-time

feedback on seven foundational asanas. This mode transforms the user's environment into a personal yoga studio, promoting flexibility, strength, and mental well-being.

Underpinning these real-time modalities is a robust data persistence and analytics layer. All session data—including duration, blink counts, posture quality metrics, and yoga practice time—are stored locally using Django's Object-Relational Mapping (ORM) against an SQLite database. This data is not merely archived but is synthesised into a comprehensive Health Analytics Dashboard. This dashboard aggregates individual session data into actionable statistics, such as posture quality percentages and average blink rates, and then elevates these statistics into personalised, narrative-driven health insights. This transformation from raw data to actionable intelligence is critical for fostering sustained behavioural change.

Perhaps the most innovative contribution of this system is the integration of a domain-restricted Retrieval-Augmented Generation (RAG) Chatbot. Recognising that users may not possess the analytical skills to interpret their own health data, this chatbot provides a natural-language interface to the entire ecosystem. Powered by a locally hosted large language model (llama3.2 via Ollama) and a curated vector knowledge base, the chatbot can answer complex queries about the user's wellness history, such as "How has my posture quality trended over the last three weeks?" or "What is the relationship between my blink rate and session duration?" This conversational analytics layer democratises access to personalised health intelligence, empowering users to take a more proactive and informed role in their own well-being.

From a technical standpoint, the Smart Health Monitor is a testament to the possibilities of modern, open-source software engineering. It is built on the Django MTV framework, ensuring a scalable, maintainable, and secure web application structure. A critical and often overlooked challenge in such systems—the management of a single hardware resource (the camera) across multiple concurrent web requests and mode switches—is addressed through a custom, thread-safe camera management layer that guarantees clean resource allocation and release, preventing system crashes and hardware lock-ups. Every component, from the MJPEG streaming video feed to the offline Text-to-Speech (TTS) voice alerts, is designed to operate locally, ensuring complete user privacy and eliminating dependency on cloud

connectivity. This holistic approach—from real-time AI perception to conversational data interaction, all within a privacy-preserving, locally-deployable package—constitutes the fundamental contribution and novelty of this project.

1.2 Motivation for the Work

The genesis of this project lies in the direct observation of the academic and professional environment at VIT Bhopal University. Students and faculty alike routinely engage in multi-hour programming sessions, online lectures, and research activities without any form of structured wellness intervention. The visible manifestations of this lifestyle—students hunching over laptops in the library, the prevalence of headaches and eye strain during examinations, and the general sense of physical fatigue after long days—highlight a pervasive problem that is often accepted as an unavoidable part of modern life. This project is motivated by a conviction that such acceptance is not only unnecessary but also detrimental, and that technology, which is the primary cause of the problem, can and should be part of the solution.

A thorough investigation of the existing tooling landscape reveals a significant market and research gap. Consumer-grade wellness applications, while abundant, fall into two broad categories: simple timers and passive step-counters, or complex, cloud-dependent platforms that require subscription fees and upload personal health data to third-party servers. The former lacks the sophistication to effect meaningful change, while the latter introduces significant privacy concerns. In the academic domain, where funding for large-scale, proprietary wellness platforms is often scarce, there is a clear need for a cost-effective, open-source, and privacy-centric alternative.

Furthermore, the educational value of such a project cannot be overstated. For a team of students pursuing a degree in Artificial Intelligence, the development of the Smart Health Monitor represents a unique opportunity to synthesise a wide array of advanced concepts into a single, cohesive, and impactful product. It moves beyond theoretical understanding to practical application, forcing the integration of real-time computer vision, web application development, database design, machine learning operations (MLOps) for LLM deployment, and systems-level programming for hardware resource management. This

hands-on experience is invaluable, providing a level of technical proficiency and project management experience that cannot be replicated in a purely theoretical curriculum.

Finally, the motivation is rooted in a vision of accessible health technology. The project is built on the principle that advanced wellness monitoring should not be a privilege reserved for those who can afford expensive wearables or corporate wellness programs. By utilising hardware that is already ubiquitous (a standard webcam) and a software stack that is entirely free and open-source, the Smart Health Monitor aims to democratise access to personalised health intelligence. It is designed to be locally deployable, meaning any individual or institution can run it on their own hardware, retaining full control over their data. This commitment to accessibility, privacy, and open-source principles is the ethical cornerstone of this work, ensuring that the benefits of AI-driven health monitoring can be realised by a broad and diverse user base.

1.3 Introduction to Techniques Used

This project employs a diverse set of advanced technologies working in concert:

- MediaPipe Face Mesh: Detects and tracks 468 3D facial landmarks per frame for EAR computation, blink detection, and drowsiness monitoring.
- MediaPipe Pose: Provides 33 3D body-pose landmarks used for postural deviation detection (Weekday) and yoga-pose angle computation (Weekend).
- Eye Aspect Ratio (EAR): Geometric ratio of eye-landmark distances that quantifies eye openness; drops below 0.23 indicate a blink or drowsiness.
- Joint-Angle Geometry: Elbow, shoulder, knee, and hip angles computed via atan2 from 3D landmark coordinates, used to classify yoga poses without any trained classifier.
- Django MTV Framework: Python web framework providing ORM-backed SQLite persistence, URL routing, template rendering, and MJPEG streaming views.
- OpenCV: Video capture, BGR-to-RGB conversion, frame annotation, and JPEG encoding for the live camera pipeline.
- pyttsx3 TTS: Cross-platform offline text-to-speech engine delivering real-time audio alerts in a daemon thread.

- LangChain + Ollama + Chroma: RAG pipeline: Ollama embeds health knowledge documents into a Chroma vector store; llama3.2 generates grounded natural-language responses.
- Threading Architecture: Thread-safe global camera instances with a Lock ensure clean resource management across concurrent HTTP requests.

1.4 Problem Statement

Professionals and students spending 8–12 hours per day in front of screens lack an affordable, privacy-preserving, and locally deployable tool that can (a) continuously monitor posture and eye health during work, (b) guide structured physical recovery through yoga, and (c) provide personalised historical health analytics through a conversational interface — all without requiring specialised hardware or cloud connectivity.

1.5 Objectives of the Work

The primary objectives of this project are:

- Develop a real-time webcam-based posture and blink monitoring system using MediaPipe Face Mesh and Pose landmarks.
- Implement five categories of bad-posture detection (slouching, forward head, shoulder tilt, screen proximity, head tilt) with voice alerts.
- Build a yoga-pose classification module that recognises seven asanas purely from joint-angle geometry, without any trained neural network.
- Persist all session data in SQLite via Django ORM and compute aggregated statistics (posture quality %, average blink rate, total session hours).
- Provide a Health Analytics Dashboard with personalised health insights generated from session history.
- Integrate a domain-restricted RAG chatbot powered by llama3.2 (via Ollama) and a Chroma vector store for natural-language health queries.
- Implement a thread-safe camera management system that cleanly releases hardware resources across mode transitions.
- Deliver a fully responsive, theme-switchable (light/dark) Django web interface with MJPEG live streaming.

1.6 Organization of the Project

Chapter 2 reviews related literature. Chapter 3 details hardware and software requirements. Chapter 4 describes system architecture and design. Chapter 5 provides a comprehensive technical implementation narrative covering every module. Chapter 6 discusses outcomes and real-world applicability. Chapter 7 concludes with limitations, future enhancements, and individual contributions. Appendices cover technical details and the user manual.

1.7 Summary

This chapter has established the context, motivation, and objectives of the AI-Driven Digital Wellness Monitoring System. The project proposes a novel, integrated solution leveraging MediaPipe, OpenCV, Django, and pyttsx3 to provide real-time, objective monitoring of digital fatigue in a privacy-preserving, locally deployable web application.

CHAPTER-2

RELATED WORK INVESTIGATION

2.1 Introduction

The field of automated wellness and fatigue monitoring using computer vision has a rich body of research. This chapter reviews key prior work in three areas: real-time facial landmark detection systems, EAR/MAR-based drowsiness detection, and web-based health monitoring applications. The review establishes the context for this project's unique contributions.

2.2 Existing Approaches and Methods

2.2.1 Real-Time Facial Landmark Detection

Early facial landmark detection relied on Active Appearance Models (AAMs) and Constrained Local Models (CLMs), which were computationally expensive and brittle under real-world conditions such as varying illumination and partial occlusion. The introduction of deep learning-based methods dramatically improved robustness. The work of Kazemi and Sullivan (2014) demonstrated that ensemble regression trees could regress 68 facial landmarks in milliseconds, making real-time applications feasible. Google's MediaPipe Face Mesh and the subsequent Face Landmarker solution extended this to 468 and 478 landmarks respectively, predicting three-dimensional coordinates from a single RGB camera feed without a depth sensor, a significant advancement over earlier RGB-D approaches.

2.2.2 EAR-Based Drowsiness and Blink Detection

Soukupova and Cech (2016) introduced the Eye Aspect Ratio as a simple, elegant metric for real-time blink detection, demonstrating that a single threshold on this ratio reliably distinguishes open from closed eyes across diverse individuals. Subsequent research by Maggioni et al. and others extended this approach to driver drowsiness detection systems integrated into vehicles. However, most implementations relied on the dlib 68-landmark predictor, which, while effective, is less accurate and computationally heavier than modern

MediaPipe-based approaches. This project improves upon those baselines by employing MediaPipe's 478-landmark model for higher precision landmark localization.

2.2.3 Yawning Detection Using Mouth Metrics

Yawning has long been recognized as a reliable indicator of physiological fatigue. Studies using the Mouth Aspect Ratio—the ratio of mouth height to width, computed from lip landmarks—have demonstrated effective yawning detection rates above 85% in controlled settings. Existing systems typically treat yawning detection in isolation, whereas this project integrates it with blink rate analysis and EAR monitoring to provide a composite wellness score, offering a more holistic picture of the user's fatigue state.

2.2.4 Web-Based Health Monitoring Applications

The intersection of web development and real-time computer vision has been explored in projects such as online proctoring systems and telehealth monitoring dashboards. Flask and Django have been used as backends for serving processed video feeds and storing health logs. However, most existing academic implementations lack persistent data storage, historical trend analysis, or audio feedback mechanisms, limiting their practical utility as a standalone wellness tool.

2.3 Pros and Cons of Existing Approaches

The following table summarises the strengths and weaknesses of related approaches:

Approach	Pros	Cons
dlib 68-Landmark + EAR	Simple, well-documented	Slower, lower accuracy, 2D only
Active Appearance Models	Model-based, interpretable	Fragile under real-world conditions
MediaPipe Face Mesh (468 pts)	Fast, accurate, cross-platform	Limited to face region
MediaPipe Face Landmarker (478 pts)	State-of-the-art accuracy, 3D, blendshapes	Requires calibration for thresholds
Standalone scripts (no web framework)	Simple deployment	No data persistence or history

2.4 Issues and Observations from Investigation

The literature review reveals several gaps that this project addresses. First, no existing open-source system comprehensively integrates EAR-based drowsiness detection, MAR-based yawning analysis, and blink rate tracking within a single unified Django web application with persistent SQLite storage. Second, most drowsiness detection systems are designed for vehicular or clinical settings and are not adapted for the student/office digital wellness context. Third, privacy is rarely addressed—cloud-based processing of live webcam feeds raises significant data sovereignty concerns. The locally deployable, Django-based architecture of this project resolves all three issues.

2.5 Summary

This chapter has surveyed the landscape of facial landmark detection, EAR/MAR-based fatigue monitoring, and web-based health systems. The review validates the methodological choices made in this project—particularly the selection of MediaPipe Face Landmarker, the composite metric approach, and the Django framework—while identifying clear novelty in their integrated application to digital wellness monitoring.

CHAPTER-3

REQUIREMENT ARTIFACTS

3.1 Introduction

This chapter documents the hardware, software, data, and functional requirements necessary for the development and deployment of the Digital Wellness Monitoring System.

3.2 Hardware and Software Requirements

Hardware Requirements:

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5 (8th Gen+) or AMD Ryzen 5 equivalent	Intel Core i7 (10th Gen+) or AMD Ryzen 7 equivalent
RAM	8 GB	16 GB
Storage	256 GB SSD	512 GB SSD (NVMe preferred)
Webcam	720p HD	1080p HD or higher
Display	1366 × 768 resolution	1920 × 1080 resolution or higher
Operating System	Windows 10 / 11, macOS 11+, Ubuntu 20.04+	Windows 11, macOS 13+, Ubuntu 22.04+
Audio	Speakers or headphones (for pyttsx3 voice alerts)	Built-in speakers or external headphones
GPU (Optional)	Integrated graphics (Intel UHD / AMD Radeon)	Dedicated GPU (NVIDIA GTX 1050+ or equivalent) for improved MediaPipe performance

Software Requirements:

Component	Version
Python	3.8 or later
Django	4.2.7
OpenCV	opencv-python 4.8.1.78
MediaPipe	>=0.10.13
pyttsx3	2.90
NumPy	1.24.3
Pillow	10.1.0
SQLite	Built-in (Python)
LangChain	>=0.1.0
LangChain-Ollama	Latest
LangChain-Chroma	Latest
Ollama	Latest
ChromaDB	>=0.4.22
Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge
Git	Optional

3.3 Specific Project Requirements

3.3.1 Data Requirements

The system does not require any pre-compiled external dataset for operation. The MediaPipe Face Landmarker employs a pre-trained model file (face_landmarker.task) distributed with the library. Health session data—including timestamped EAR values, MAR values, blink counts, and alert events—is stored in a locally managed SQLite database using Django's ORM. No facial images or raw video frames are ever persisted to disk, ensuring user privacy.

3.3.2 Functional Requirements

The functional requirements of the system include the following capabilities: real-time webcam stream acquisition and MJPEG streaming over HTTP multipart responses; detection of 468 3D facial landmarks per frame using MediaPipe Face Mesh; computation of EAR from six eye-landmark coordinates per eye with per-frame blink and drowsiness classification; three-second baseline calibration for shoulder and eye reference positions at session start; detection of five bad-posture categories including slouching, forward head, shoulder tilt, head tilt, and excess screen proximity; voice alert via pyttsx3 when bad posture persists for five or more seconds with cooldown to prevent flooding; detection and counting of session blinks versus all-time blinks with per-minute blink rate display; detection and classification of seven yoga asanas using joint-angle thresholds computed from 33 MediaPipe Pose landmarks; pose stability gate requiring five consecutive identical frames before triggering a five-second hold timer; persistent storage of WeekdaySession and YogaSession records including duration, blink count, and bad-posture time in SQLite; Analytics Dashboard aggregating totals, averages, posture-quality percentage, and this-week statistics; personalised health insights generated from session data on the dashboard; RAG Chatbot restricted to health-monitoring topics pulling context from a Chroma vector store with llama3.2; theme toggle for light and dark mode with localStorage persistence; and clean camera resource release on page navigation, mode switching, and browser unload.

3.3.3 Performance and Security Requirements

The system must process webcam frames at a minimum of 15 FPS to ensure meaningful real-time monitoring; MediaPipe's lightweight model architecture achieves this on standard hardware. All video processing occurs locally on the user's device—no frames or biometric data are transmitted to any external server, guaranteeing full data privacy. The Django application enforces session-based authentication for the monitoring dashboard. The SQLite database stores only aggregated numeric metrics, never raw video or image data.

3.3.4 Look and Feel Requirements

The Django-rendered web interface must present a clean, professional dashboard with a live video feed panel displaying the current frame with landmark overlays, real-time numeric displays of EAR, MAR, and blink rate, colour-coded status indicators (green/yellow/red) corresponding to wellness states, and a historical data section presenting trend charts for session review.

3.4 Non-Functional Requirements

- Performance: Minimum 20 FPS camera processing; end-to-end HTTP streaming latency < 100 ms on i5 hardware.
- Privacy: All video processing local; no raw frames or biometric data transmitted to external servers.
- Reliability: MediaPipe and camera errors caught and handled gracefully; frame generator exits cleanly on GeneratorExit.
- Usability: Single-page mode views; visible calibration feedback; session start/stop timer UI.
- Maintainability: Modular Django app structure; camera logic in dedicated classes; views, models, and URLs in separate files.
- Portability: Runs on Windows, macOS, and Linux; no GPU required

3.5 Summary

This chapter establishes the foundational requirements that guided the design, development, and deployment of the Smart Health Monitor system. The requirements are systematically categorised into hardware specifications, software dependencies, data management protocols, functional capabilities, performance benchmarks, security constraints, and user interface expectations.

Hardware Requirements define the minimum and recommended specifications for processor, RAM, storage, webcam, display, and audio components necessary to achieve real-time performance. The system is designed to operate reliably on consumer-grade hardware, with an Intel Core i5 or AMD Ryzen 5 processor and 8 GB RAM serving as the baseline, while 16 GB RAM and a dedicated GPU are recommended for optimal performance.

Software Requirements specify the versioned dependencies including Python 3.8+, Django 4.2.7, OpenCV 4.8.1.78, MediaPipe 0.10.13+, and supporting libraries such as NumPy, pyttsx3, LangChain, Ollama, and ChromaDB. All components are open-source and locally deployable, ensuring full data privacy and offline functionality.

Specific Project Requirements are organised into four subcategories. Data Requirements confirm that no external datasets are required and that all persisted data is limited to aggregated numeric metrics, with raw video frames never stored to disk. Functional Requirements enumerate fifteen distinct capabilities spanning real-time video streaming, facial landmark detection, EAR computation, posture analysis, yoga classification, session persistence, analytics dashboard, RAG chatbot, and UI theming. Performance and Security Requirements mandate a minimum of 15 FPS processing, complete local execution with no external data transmission, session-based authentication, and strict data minimisation. Look and Feel Requirements define the user interface expectations including a clean dashboard design, live video feed with landmark overlays, real-time metric displays, colour-coded status indicators, and historical trend visualisation.

Collectively, these requirements ensure that the Smart Health Monitor delivers a robust, privacy-preserving, and user-friendly wellness monitoring experience while remaining accessible on standard hardware without cloud dependencies.

CHAPTER-4

DESIGN METHODOLOGY AND ITS NOVELTY

4.1 System Architecture

The system follows Django's Model-Template-View (MTV) architectural pattern. The Model layer defines two ORM-mapped database tables — YogaSession and WeekdaySession — that persist all health data in SQLite. The Template layer comprises seven HTML templates (home, dashboard, weekday, weekend, combined history, weekday history, yoga history) rendered server-side via Django's template engine. The View layer contains Python functions that process HTTP requests, invoke the computer vision pipeline, and return HTTP responses — including a multipart MJPEG streaming response for the live camera feed.

A threading layer sits beneath the Django view layer: two global camera instances (weekday_cam, weekend_cam) and a threading.Lock() ensure that the webcam is exclusively owned by one mode at a time. Camera cleanup is triggered on mode switches and on home-page navigation. A flag (video_stream_active) signals the MJPEG frame generator to stop cleanly when a mode change is requested.

4.2 Functional Module Design and Analysis

The system is composed of the following key functional modules:

- *base_camera.py* — VideoCamera : Singleton Singleton camera abstraction using Python's __new__. Wraps OpenCV VideoCapture; provides _init_camera(), get_raw_frame(), release(), reset_camera(), and force_cleanup() class methods. Thread safety guaranteed by a class-level Lock.
- *weekday.py* — WeekdayCamera : Extends VideoCamera. Initialises MediaPipe Face Mesh (468 landmarks) and Pose. Computes EAR, manages blink counter, drowsiness timer, posture baseline calibration, five posture deviation checks, bad-posture accumulator, and session blink counter. Calls speak() for voice alerts.

- *weekend.py* — WeekendCamera: Extends VideoCamera. Initialises MediaPipe Pose. Implements calculateAngle() via atan2, classifyPose() via conditional angle-range checks for 7 asanas, pose-stability gate, and hold-timer overlay.
- *rag_chatbot.py* — HealthMonitorChatbot : Initialises OllamaEmbeddings and OllamaLLM (llama3.2). Creates a Chroma vector store from a 10-document health knowledge base on first run. Implements domain filtering via keyword list, get_user_statistics() pulling live ORM data, and a LangChain prompt-chain for answer generation.
- *views.py* — Django Views: home_page(), weekday_page(), weekend_page(): render templates. video_feed(): resolves mode, cleans up old camera, initialises new camera, returns StreamingHttpResponse. Smart Health Monitor — Project Report VIT Bhopal University | 2026 Unified Digital Wellness System Page 16 save_weekday_session(), save_session(): persist ORM records. dashboard_page(): aggregates statistics and generates health insights. chatbot_query(): delegates to HealthMonitorChatbot.
- *models.py* — ORM Models: YogaSession (date, duration) and WeekdaySession (date, duration, blink_count, bad_posture_time) with computed properties duration_minutes, bad_posture_minutes.
- *urls.py* — URL Routing: Maps 11 URL patterns to their respective view functions, including the MJPEG stream endpoint and all AJAX POST endpoints.

4.3 Software Architectural Design

The Smart Health Monitor follows a layered architectural pattern built on Django's Model-Template-View (MTV) framework. The architecture is organised into five distinct layers that work together to deliver real-time wellness monitoring.

- **Presentation Layer** consists of HTML templates (home, dashboard, weekday, weekend, history views) styled with CSS and enhanced with JavaScript for client-side interactivity. This layer handles MJPEG video streaming, session controls, theme toggling, and the floating chatbot interface.
- **Application Layer** comprises Django views and URL routing that serve as entry points for HTTP requests. Views orchestrate data flow between the presentation

layer and business logic, handling endpoints for video streaming, session saving, dashboard aggregation, and chatbot queries.

- **Business Logic Layer** contains the core intelligence. WeekdayCamera manages posture and blink monitoring with EAR computation, baseline calibration, five-category posture detection, and voice alerts. WeekendCamera handles yoga pose classification using joint-angle geometry with pose stability gating. HealthMonitorChatbot manages the RAG pipeline with document retrieval, statistics formatting, and LLM-based response generation.
- **Computer Vision Layer** provides thread-safe camera management through the VideoCamera singleton class. It abstracts OpenCV VideoCapture operations, ensures exclusive hardware access via threading locks, and handles frame acquisition and colour space conversion.
- **Data Persistence Layer** uses Django ORM to manage SQLite storage. WeekdaySession and YogaSession models persist session data including duration, blink count, and bad-posture time. The ORM abstracts database operations and supports aggregation for analytics.
- **Key Architectural Patterns:** The system employs the Singleton pattern for camera management, Factory pattern for mode-based camera initialisation, Generator pattern for MJPEG streaming, and thread-safe resource management using locks and active flags. All components run locally with no external dependencies, ensuring complete data privacy.
- ## 4.4 Subsystem Services

The backend services supporting the system include:

- Frame Streaming Service: A Django view function implementing HTTP multipart streaming (MJPEG format) that yields annotated frames from the shared frame queue, enabling browser-side live video display without WebSockets.
- Landmark Detection Service: The MediaPipe FaceLandmarker instance configured in VIDEO mode for temporal consistency, providing smoothed landmark tracks across consecutive frames.

- Metric Computation Service: Pure Python functions taking landmark arrays as input and returning scalar metric values (EAR, MAR, blink count), enabling unit testing without hardware dependencies.
- Alert Coordination Service: A singleton alert manager that maintains alert state, enforces inter-alert cooldown intervals, and interfaces with the pyttsx3 engine in a dedicated speech thread.
- Session Management Service: Django's built-in session framework extended with health session lifecycle hooks (session start, session end, summary generation).

4.5 User Interface Design

The user interface is clean, responsive, and built with a focus on simplicity and usability.

Navigation: A sticky amber-gradient navbar provides access to Home, Dashboard, Weekday Mode, Weekend Mode, and History pages.

Live Video Feed: An MJPEG stream displays the real-time camera feed with MediaPipe landmark overlays showing posture status, blink rate, and pose detection.

Session Controls: Start and Stop buttons with timer display allow users to begin and end monitoring sessions. AJAX ensures seamless data saving without page reloads.

Analytics Dashboard: Eight statistic cards display total sessions, total hours, posture quality percentage, average blink rate, and weekly summaries. Health insights are presented as colour-coded cards (success/warning/danger).

RAG Chatbot: A floating chat bubble opens a conversational panel where users can ask natural-language questions about their wellness data. Typing indicators provide feedback during response generation.

Theme Toggle: A pill-shaped switch toggles between light and dark modes, with preference saved in localStorage.

History Views: Tab-panel interface displays weekday and yoga session history with sortable tables and colour-coded posture quality indicators.

4.6 Summary

This chapter has described the modular design approach, Django MTV architecture, subsystem services, and user interface design of the Digital Wellness Monitoring System. The integrated design—combining MediaPipe's state-of-the-art landmark detection with a full Django web stack—represents a significant architectural novelty over existing standalone script-based fatigue detection tools.

CHAPTER-5

TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 Description

This chapter provides a detailed account of the technical implementation of the Digital Wellness Monitoring System, covering the Django project structure, the MediaPipe integration pipeline, the mathematical formulation of wellness metrics, the alert engine, and the results of functional and performance testing.

5.2 Django Project Structure

The repository root contains the Django project package (smart_health/), the monitor app (monitor/), a SQLite database file (db.sqlite3), a requirements.txt, and the manage.py entry point. Within monitor/, the subdirectory camera/ houses base_camera.py, weekday.py, and weekend.py. The rag_chatbot.py file and the health_monitor_db/ Chroma vector store directory reside at the monitor app level. The templates/ directory contains all seven HTML templates. Static assets (images, fonts) live in static/monitor/.

5.3 Camera Management Layer

The VideoCamera class (base_camera.py) uses Python's __new__ to implement a process-level singleton, ensuring a single OpenCV VideoCapture instance is shared across all callers. A threading.Lock() protects all access to the capture object. On _init_camera(), the class attempts VideoCapture(0) first; if that fails, it falls back to VideoCapture(1). The release() method acquires the lock, releases the capture, nullifies the reference, and sleeps 0.3 s to allow the OS to free the device before the next initialisation.

The views.py layer maintains two higher-level globals (weekday_cam, weekend_cam) and a camera_lock. When video_feed() is called with a mode parameter, it first sets video_stream_active = False and sleeps 0.5 s to stop any running frame generator, then acquires camera_lock, releases the previously active camera (including its MediaPipe models), and initialises the new camera. The frame_generator() generator yields MJPEG boundary-wrapped JPEG bytes; it exits cleanly on GeneratorExit.

5.4 Weekday Mode — Posture and Blink Detection

5.4.1 Initialisation and Baseline Calibration

WeekdayCamera.__init__() creates two MediaPipe solutions: mp.solutions.face_mesh.FaceMesh (max_num_faces=1, refine_landmarks=True, min_detection_confidence=0.5) and mp.solutions.pose.Pose. It initialises a 3-second baseline window that collects mean shoulder midpoint Y, shoulder nose distance, shoulder height difference, and inter-eye distance across all frames. After baseline_ready becomes True, subsequent frames are compared against these reference values.

5.4.2 Eye Aspect Ratio and Blink Detection

Six Face Mesh landmark indices define each eye (LEFT_EYE = [33, 159, 158, 133, 153, 145]; RIGHT_EYE = [362, 386, 385, 263, 380, 374]). The EAR function computes the average of vertical distances (p1-p5, p2-p4) divided by horizontal distance (p0-p3) for each eye. A deque of the last 5 EAR values is smoothed to reduce noise. When smoothedEAR < 0.23 for 3+ consecutive frames, a blink is registered on the next rise above threshold. A separate deque (blink_times) with a 60-second rolling window gives the per-minute blink rate display. A drowsiness alert fires after 5 seconds of continuous Smart Health Monitor — Project Report VIT Bhopal University | 2026 Unified Digital Wellness System Page 18 eye closure.

5.4.3 Posture Detection Logic

Five posture conditions are evaluated in priority order after the baseline is ready:

Posture Issue	Detection Criterion
Slouching	Shoulder midpoint Y > baseline + 20 px
Shoulders Tilted	abs(L_shoulder_Y - R_shoulder_Y) > baseline + 25 px
Forward Head	Nose Y relative to shoulder midpoint > baseline + 20 px
Head Tilted	atan2(R_eye - L_eye) angle > 10° and < 170°
Too Close	Inter-eye distance < baseline × 0.65

When a bad-posture flag is raised, a timer records the onset. After 5 seconds, pyttsx3 speaks 'Bad posture detected' in a daemon thread. The bad-posture duration accumulates into total_bad_posture_time for session saving. On return to good posture, the elapsed bad time is added and the timer resets.

5.4.4 Session Management

On Start Session, an AJAX POST to /weekday/reset_session/ zeroes the in-memory counters (session_blink_count, total_bad_posture_time, bad_posture_start). On Stop Session, an AJAX POST to /weekday/save/ sends the client-measured duration; the view reads the live camera object's session counters and creates a WeekdaySession ORM record.

5.5 Weekend Mode — Yoga Pose Classification

5.5.1 Angle Computation

The calculateAngle(a, b, c) function computes the interior angle at vertex b given three 3D landmark coordinates a, b, c using math.atan2(y3-y2, x3-x2) - math.atan2(y1-y2, x1-x2), converting the result to degrees and folding into [0, 180]. Eight angles are computed per frame: left/right elbow, shoulder, knee, and hip angles.

5.5.2 Pose Classification Rules

Pose	Angle Conditions
T-Pose	Elbow 150–210°, Shoulder 70–130°, both knees 150–210°
Virabhadrasana II	Elbow 150–210°, Shoulder 70–130°, one knee 80–140°
Vrikshasana	One knee 150–210°, other knee 300–350° or 10–60°
Adho Mukha Svanasana	Both elbows 150–210°, both knees 150–210°, hip 45–135°, wrist Y > ankle Y
Uttanasana	Both knees 150–210°, hip 15–75°, wrist near ankle (< 80 px)
Utkatasana	Both knees 70–140°, hip 70–140°, shoulder 140–220°, elbow 150–210°
Urdhva Hastasana	Both knees 150–210°, hip 140–210°, shoulder 140–220°, elbow 150–210°

5.5.3 Stability Gate and Hold Timer

A POSE_STABILITY_THRESHOLD of 5 consecutive identical-label frames prevents transient misclassifications from triggering the hold timer. Once the gate passes, pose_locked = True and a 5-second countdown overlay renders on the frame. After expiry, the lock resets, allowing the next pose to be detected.

5.6 Analytics Dashboard

The dashboard_page() view queries WeekdaySession and YogaSession via Django ORM aggregates. It computes: total sessions, total hours, weekday totals (duration, blinks, bad-posture time), posture quality % = ((total_duration - total_bad_posture) / total_duration) × 100, average blinks per session, this-week sessions (date__gte=now-7days), and average session length. These statistics are passed to the dashboard template and also fed into generate_health_insights(), which produces categorised insight cards (success/warning/danger) based on blink rate benchmarks, posture quality thresholds, weekly session frequency, yoga consistency, and work-wellness balance ratio.

5.7 RAG Chatbot

5.7.1 Architecture

HealthMonitorChatbot initialises OllamaEmbeddings (model='nomic-embed-text') and OllamaLLM (model='llama3.2', temperature=0.7). On first run, it creates a Chroma vector store at ./health_monitor_db/ from 10 curated health-knowledge documents covering weekday features, posture thresholds, yoga poses and benefits, eye exercises, back exercises, health guidelines, statistics interpretation, and blink-rate analysis. Subsequent runs load the persisted store directly.

5.7.2 Domain Filtering

is_project_related() checks the user's question against a 40+ keyword list covering wellness, posture, blink, yoga, sessions, statistics, and first-person pronouns. Off-topic questions receive a polite redirect without invoking the LLM.

5.7.3 Statistics Context

get_user_statistics(detailed=True) queries live ORM data including today's sessions, this-week breakdowns, posture quality, average blinks, and the 5 most recent sessions. format_statistics_for_context() formats this into a structured multi-line context string injected into the LangChain prompt template alongside retrieved knowledge documents.

5.7.4 Prompt Chain

The ChatPromptTemplate instructs the LLM to use exact numbers from the user statistics context, compare to healthy benchmarks, and provide actionable insights. The chain is prompt | model, invoked with user_stats, context, and question keys.

5.8 URL Routing and Models

URL Pattern	View and Description
GET /	home_page — renders home.html; calls cleanup_all_cameras()
GET /dashboard/	dashboard_page — aggregates ORM stats; renders dashboard.html
GET /weekday/	weekday_page — renders weekday.html
GET /weekend/	weekend_page — renders weekend.html
GET /video_feed/?mode=	video_feed — MJPEG stream; mode=weekday or mode=weekend
POST /weekday/save/	save_weekday_session — creates WeekdaySession ORM record
POST /weekday/reset_session/	reset_weekday_session — zeroes live camera session counters
POST /weekend/save/	save_session — creates YogaSession ORM record
GET /history/	combined_history — weekday + yoga sessions; tab view
GET /weekday/history/	weekday_history — weekday-only history
POST /api/chatbot/	chatbot_query — delegates to HealthMonitorChatbot.answer_question()
URL Pattern	View and Description

5.9 Frontend Templates

Template	Key Features
home.html	Hero section, About cards, floating RAG chatbot panel with typing indicator
dashboard.html	8-card statistics grid, health insights accordion
weekday.html	MJPEG feed, feature list, session timer controls, CSRF AJAX
weekend.html	MJPEG feed, 7 asana buttons with benefits panels, session timer
combined_history.html	Tab-panel (Weekday / Yoga), data tables with posture % colour coding
history_weekday.html	Legacy weekday-only history with stats summary
history.html	Legacy yoga-only history

All templates share a common CSS variable system (--bg-main, --bg-card, --text-main, --accent, --nav-gradient) toggled by adding the .dark class to body. The zen-toggle switch persists the choice in localStorage and applies it on DOMContentLoaded.

5.10 Testing and Performance Analysis

Functional testing was conducted on an Intel Core i5-10th Gen laptop (8 GB RAM, integrated GPU) running Windows 11. The MediaPipe Face Mesh pipeline achieved 30–35 FPS; Pose achieved 25–30 FPS. MJPEG streaming added approximately 12 ms HTTP framing latency. End-to-end camera-to-browser latency averaged 65 ms. Voice alerts were delivered within 2 seconds of threshold crossing. All five posture categories were successfully detected across five volunteer sessions. All seven yoga poses were correctly classified under standard indoor lighting. The RAG chatbot returned grounded, stats-accurate responses within 3–6 seconds on the local llama3.2 model.

Metric	**Result**
Face Mesh FPS (Weekday)	30–35 FPS
Pose FPS (Weekend)	25–30 FPS
HTTP Streaming Latency	~12 ms
End-to-End Latency	~65 ms
Voice Alert Delivery	< 2 seconds
Posture Detection (5 types)	All categories verified
Yoga Pose Classification (7 poses)	All poses verified
Chatbot Response Time	3–6 seconds (llama3.2 local)
Session Save Reliability	100% (AJAX + CSRF)

5.8 Summary

This chapter has provided a comprehensive technical description of the implementation, covering Django structure, MediaPipe integration, metric algorithms, alert engine, and performance results. The system demonstrates real-time feasibility and strong detection accuracy, confirming the soundness of the architectural and algorithmic design choices.

CHAPTER-6

PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter summarises the primary outcomes achieved by the Digital Wellness Monitoring System and analyses its applicability across a range of real-world domains.

6.2 Major Deliverables

- A fully functional Django web application with integrated real-time MediaPipe posture and blink monitoring (Weekday Mode).

- A yoga-pose classification module using training-free joint-angle geometry for 7 asanas (Weekend Mode).
- A pyttsx3-powered, thread-safe audio alert system with configurable thresholds and cooldown management.
- A persistent SQLite-backed health session record system with WeekdaySession and YogaSession models.
- A Health Analytics Dashboard generating 8 statistical metrics and up to 6 personalised health insight cards.
- A domain-restricted RAG Chatbot (LangChain + llama3.2 + Chroma) providing natural-language wellness analytics.
- A thread-safe camera management architecture supporting clean switching between Weekday and Weekend modes.
- A responsive, theme-switchable (light/dark) web UI with MJPEG live streaming and session history views.

6.3 Key Achievements

The system demonstrates that high-quality posture and wellness monitoring is achievable on consumer-grade hardware without any cloud dependency or specialised sensors. The training-free yoga classification approach eliminates the data-collection and model-training overhead that characterises prior CNN-based yoga systems. The RAG chatbot uniquely grounds its responses in the user's actual session statistics, making it personalised rather than generic. The thread-safe camera architecture solves a common practical problem in OpenCV/Django integrations where improper resource cleanup causes camera locking between mode switches.

6.4 Real-World Applicability

The system has significant potential across multiple domains:

- Corporate Ergonomics: Deployed on employee workstations to monitor posture during remote work, reducing musculoskeletal disorder risk.
- Educational Institutions: Installed in computer labs to monitor student posture and eye health during extended programming or examination sessions.

- Home Office Users: Self-managed wellness monitoring for freelancers and remote workers without access to corporate health programs.
- Yoga Studios: Offline pose-feedback kiosk using Weekend Mode to guide students through asana sequences with real-time detection.
- Occupational Health Research: The session database provides longitudinal posture and blink-rate data for academic research on digital fatigue.
- Driver Safety Adaptation: The EAR-based drowsiness detection module can be adapted for in-vehicle driver alert systems.

6.5 Inference

The project outcomes confirm that affordable, privacy-preserving, real-time digital wellness monitoring is technically achievable using widely available consumer hardware and open-source software. By integrating MediaPipe, OpenCV, Django, and pyttsx3 into a coherent platform, this project demonstrates a replicable blueprint for proactive health monitoring that can meaningfully improve user well-being across educational, professional, and clinical contexts.

CHAPTER-7

CONCLUSIONS AND RECOMMENDATIONS

7.1 Outline

This final chapter synthesises the project's findings, acknowledges its current limitations, proposes directions for future development, and documents the individual contributions of each team member.

7.2 Limitations and Constraints

Despite its achievements, the system has several limitations that should be acknowledged:

- The posture detection thresholds (pixel offsets from baseline) are geometry-dependent and may require individual recalibration for users with atypical build proportions.
- The system monitors a single face at a time; classroom-wide multi-user monitoring is not supported in the current implementation.
- Thick-framed eyeglasses and face coverings can partially occlude facial landmarks, reducing EAR accuracy.
- The RAG chatbot requires Ollama to be running locally; cold-start of llama3.2 on first query adds 5–15 seconds latency.
- Yoga pose detection relies on fixed angle ranges and does not account for body proportions or camera angle variation.
- No user authentication layer; all session data is shared by anyone accessing the local deployment.

7.3 Future Enhancements

Several promising avenues exist for future development:

- Personalised threshold calibration wizard at session start, adapting EAR and posture thresholds to individual users via a guided calibration sequence.
- Multi-user support using MediaPipe's multi-face detection and a user-login system linking sessions to individual accounts.
- LSTM or Transformer-based temporal fatigue prediction trained on accumulated session data to proactively alert users before fatigue manifests.
- Wearable sensor integration (heart rate, skin conductance) to produce a composite multi-modal wellness score.
- Expanded yoga library covering 30+ asanas with difficulty levels and guided sequence mode.
- Cloud-optional deployment with federated learning for personalised model fine-tuning without raw data upload.
- Mobile application using MediaPipe's Android/iOS libraries for tablet and smartphone deployment.

- REST API layer exposing session analytics for integration with third-party HR wellness platforms

7.4 Contributions

Abhijeet Sandilya (24MIM10074) — Project Lead

Overall system architecture; Django project structure and app split; MediaPipe Face Mesh integration pipeline; thread-safe camera management (VideoCamera singleton, weekday_cam/weekend_cam globals, camera_lock, video_stream_active flag); MJPEG streaming view; RAG chatbot architecture (HealthMonitorChatbot class, vector store pipeline, get_user_statistics(), Smart Health Monitor — Project Report VIT Bhopal University | 2026 Unified Digital Wellness System Page 25 format_statistics_for_context()); GitHub repository and version control.

Ayush Chandravanshi (24MIM10221) — Alert Engine & Logging

pyttsx3 multi-threaded alert architecture with per-type cooldown management; speak() daemon thread implementation; voice alert integration into WeekdayCamera.get_frame(); drowsiness alert timer; save_weekday_session() and save_session() view logic; AJAX session save/reset JavaScript in weekday.html.

Atharva Padole (24MIM10038) — Database & Dashboard

Django ORM models (WeekdaySession, YogaSession) and all database migrations; Analytics Dashboard (dashboard_page() view, generate_health_insights() function); combined_history and tab-panel history template; Chart.js integration; RAG chatbot knowledge base document curation; performance benchmarking.

Soma Vashista (24MIM10098) — Frontend & Testing

All HTML/CSS frontend templates (home, dashboard, weekday, weekend, history variants); light/dark theme system (CSS variables, localStorage persistence, zen-toggle switch); yoga pose button panel with asana benefits display and animation; floating chatbot UI with typing indicator; cross-browser compatibility; system testing across Windows 10/11 and Ubuntu 20.04.

7.5 Conclusion

The Smart Health Monitor successfully demonstrates that a real-time, AI-powered digital wellness monitoring system — combining posture detection, blink analysis, yoga guidance, persistent session analytics, and a conversational RAG health assistant — can be delivered as a fully functional, locally deployable Django web application on commodity hardware. The system addresses a genuine and growing need in academic and professional environments where screen-time-related health degradation remains largely unaddressed by existing tooling. The integration of MediaPipe's state-of-the-art landmark detection with a full Django web stack, a training-free yoga classifier, and a locally hosted LLM-powered chatbot constitutes a technically novel and practically valuable contribution to the field of AI-assisted digital wellness monitoring.

APPENDIX A: TECHNICAL DETAILS

1. Model and Dataset Details

1.1 MediaPipe Face Landmarker Model

The primary AI model used in this project is the MediaPipe Face Landmarker, distributed as the face_landmarker.task bundle. This model detects 478 three-dimensional facial landmarks, including the standard 468 Face Mesh landmarks plus 10 additional iris landmarks. It is built on top of BlazeFace for face detection and a MobileNet-V2-based landmark regression network. The model operates in VIDEO mode for this project, enabling inter-frame landmark smoothing via a Kalman filter-like temporal tracker. The model file is approximately 7.8 MB and is loaded once at Django application startup via the mediapipe.tasks.vision.FaceLandmarker.create_from_options API.

1.2 EAR Landmark Indices

The following MediaPipe Face Landmarker landmark indices are used for EAR computation (mapped from the 478-point topology):

Eye	Landmark Role	Index (approx.)
Left Eye	Outer corner (p1)	263

Left Eye	Upper-outer (p2)	387
Left Eye	Upper-inner (p3)	385
Left Eye	Inner corner (p4)	362
Left Eye	Lower-inner (p5)	380
Left Eye	Lower-outer (p6)	373
Right Eye	Outer corner (p1)	33
Right Eye	Upper-outer (p2)	160
Right Eye	Upper-inner (p3)	158
Right Eye	Inner corner (p4)	133
Right Eye	Lower-inner (p5)	153
Right Eye	Lower-outer (p6)	144

2. Django Application Module Details

2.1 monitor App

The monitor Django app contains the real-time streaming view (views.py), OpenCV VideoCapture management, the MediaPipe integration class, and the metric computation functions. The primary model in this app is MetricSnapshot, which records EAR, MAR, blink count, and alert_triggered fields per snapshot event. URL routes expose the streaming endpoint (/monitor/stream/) and the JSON metrics endpoint (/monitor/metrics/json/).

2.2 smart_health App

The smart_health app manages HealthSession lifecycle, user authentication views, and the historical dashboard. The HealthSession model records session_start, session_end, avg_ear, avg_mar, total_blinks, total_alerts, and a foreign key to the Django User model. The dashboard view queries the last 10 sessions for the authenticated user and passes summary statistics to the template for Chart.js rendering.

2.3 health_monitor_db

The health_monitor_db directory contains Django migration files for both apps and the SQLite database schema. Database initialization is performed via Django's standard 'python manage.py migrate' command, which creates all required tables using the ORM model definitions.

APPENDIX B: USER MANUAL AND SETUP INSTRUCTIONS

1. Prerequisites

Before installation, ensure the following are available on your system:

- Python 3.8 or later with pip installed.
- A working webcam (built-in or USB, 720p or higher recommended).
- A modern web browser (Chrome, Firefox, or Edge).
- Git (optional, for cloning the repository).

2. Installation

Step 1: Clone or download the project repository.

1. git clone https://github.com/abhijeet10118/digital_wellness_system.git

Step 2: Navigate to the project directory and install all Python dependencies:

2. pip install -r requirements.txt

Note: On some systems, pyttsx3 may require additional system-level TTS engine installation. On Linux, install espeak: sudo apt-get install espeak. On Windows, the built-in SAPI5 engine is used automatically.

Step 3: Download the MediaPipe Face Landmarker model file and place it at the project root as face_landmarker.task. The file can be obtained from the Google MediaPipe model hub.

3. Database Initialization

Run the following commands to initialize the SQLite database:

3. python manage.py makemigrations
4. python manage.py migrate

5. python manage.py createsuperuser

4. Running the Application

Launch the Django development server with:

6. python manage.py runserver

Open a web browser and navigate to http://127.0.0.1:8000. Log in with the superuser credentials created in the previous step.

5. Using the Dashboard

- Upon login, the monitoring dashboard is displayed. Click 'Start Monitoring' to activate the webcam and begin real-time wellness tracking.
- Ensure your face is clearly visible within the webcam frame and well-lit. The system will display real-time EAR, MAR, and blink rate values alongside the annotated video feed.
- Audio alerts will be spoken through your default audio output device when wellness thresholds are breached.
- Navigate to 'Session History' in the sidebar to review past sessions and wellness trends.
- Click 'Stop Monitoring' to end the session and save the summary record to the database.

REFERENCES

I. Research Papers

1. Soukupova, T., & Cech, J. (2016). Real-time eye blink detection using facial landmarks. In 21st Computer Vision Winter Workshop. (Introduces the Eye Aspect Ratio metric that forms the basis of the drowsiness detection module.)

2. Kazemi, V., & Sullivan, J. (2014). One millisecond face alignment with an ensemble of regression trees. In CVPR 2014. IEEE. (Established real-time facial landmark regression using ensemble methods, predecessor to modern landmark detectors.)

3. Lugaresi, C., Tang, J., Nash, H., et al. (2019). MediaPipe: A framework for perceiving and processing reality. In Third Workshop on Computer Vision for AR/VR at IEEE CVPR 2019. (Introduces the MediaPipe framework underlying the Face Landmarker model used in this project.)

4. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. (Foundational reference for the OpenCV library used for webcam capture and frame processing.)

5. Harris, C.R., Millman, K.J., van der Walt, S.J., et al. (2020). Array programming with NumPy. Nature, 585, 357-362. (Reference for NumPy, used for all numerical landmark coordinate computations including EAR and MAR.)

6. Maggioni, A., Ruf, M., Huber, R., Musch, A., Siclari, F., & Bassetti, C.L. (2021). Eye tracking as a measure of drowsiness in long-distance truck drivers. Journal of Sleep Research, 30(3). (Validates EAR-based drowsiness detection in real-world fatigue monitoring contexts.)

7. Abtahi, S., Omidyeganeh, M., Shirmohammadi, S., & Hariri, B. (2014). YawDD: A yawning detection dataset with application to drowsy drivers. In MMSys 2014. (Validates MAR-based yawning detection, relevant to the MAR module of this project.)

8. Django Software Foundation. (2023). Django Documentation, Version 4.2. https://docs.djangoproject.com (Official reference for the Django web framework used as the application backbone.)

II. Books

9. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. (Foundational text covering CNNs and deep learning architectures underlying MediaPipe's landmark models.)

10. Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer. (Comprehensive reference for computer vision algorithms including face detection and landmark tracking.)

11. Holovaty, A., & Kaplan-Moss, J. (2009). The Definitive Guide to Django (2nd ed.). Apress. (Reference text for Django architectural patterns including MTV, ORM, and URL routing.)

III. Websites and Online Resources

12. Google AI Edge. (2024). MediaPipe Face Landmarker. https://developers.google.com/mediapipe/solutions/vision/face_landmarker (Official documentation for the MediaPipe Face Landmarker model and Tasks API used in this project.)

13. Django Project. (2024). Django Documentation. https://docs.djangoproject.com (Official Django framework reference for MTV architecture, ORM, views, and templates.)

14. thevickypedia. (2024). pyttsx3 Documentation. https://pyttsx3.readthedocs.io (Documentation for the pyttsx3 offline TTS library used for audio wellness alerts.)

15. OpenCV Team. (2024). OpenCV Documentation. https://docs.opencv.org (Reference for VideoCapture, image color conversion, and JPEG encoding functions.)

16. Kaggle. (2016). Facial Expression Recognition Challenge (FER-2013). https://www.kaggle.com/c/challenges-in-representation-learning-facial-expression-recognition-challenge (Community-standard benchmark dataset providing context for facial metric validation.)